

Visual Line Tracking (PTZ)

1. Experiment Objective

The camera detects the line on the ground (HSV color segmentation), PID controls the chassis angular velocity to drive along the line, and the LiDAR stops the car when an obstacle is detected ahead. The PTZ tilts down (pitch=-90°) toward the ground.

2. Experiment Principle

1. Key Terms

Term	Description
Line Tracking	The camera detects the ground line and computes the yaw error relative to the image center; a PID closed loop controls the angular velocity
HSV Color Segmentation	Binarizes the HSV color space with <code>cv.inRange</code> to extract lines of a specific color, with hue wraparound support
Tracking ROI	Processes only the lower half of the image (<code>line_roi_top_ratio</code> =0.55), excluding the distant background
Discrete PID Controller	P+I+D independent gains with integral limiting; the pixel error is scaled by <code>line_error_divisor</code>
LiDAR Obstacle Stop	Obstacle points ahead in <code>/scan</code> exceed the threshold → stop immediately + optional buzzer
Line-Lost Auto Exit	After the line-lost timeout, automatically calls <code>rc1py.shutdown()</code> to exit the node
PTZ Tilt-Down	The pan-tilt sets pitch=-90° toward the ground, more flexible than a fixed mount
QoS	best_effort (image + LiDAR + cmd_vel), sensor_data (LiDAR)
TF Coordinate Frame	odom → base_footprint → base_link

2. Technical Architecture

Dual timers (image at 30 Hz + control at 20 Hz) + LiDAR callback + PTZ tilt-down:

```

/camera/image_raw (BEST_EFFORT)
  → follow_line_ptz_node
    └─ Image processing (30 Hz): image conversion → flip → ROI HSV binarization
→ morphology → largest contour → EMA filtering
    └─ Chassis control (20 Hz): pixel error → scaling → discrete PID → angular
velocity limiting → /cmd_vel (BEST_EFFORT)
    └─ PTZ: tilts down to -90° at startup → /cmd_ptz_angle
/scan (sensor_data) → obstacle avoidance callback: valid point count → stop +
buzzer above threshold

```

Key PTZ vs. No-PTZ differences: The pan-tilt can tilt down -90° toward the ground, so there is no need to manually adjust the camera mounting angle. Line tracking performs better than the fixed-mount version.

3. Core Topics Quick Reference

Firmware Uplink

Topic	Message Type	QoS	Description
/camera/image_raw	sensor_msgs/Image	best_effort	Image from the ESP32 camera
/scan	sensor_msgs/LaserScan	sensor_data	LiDAR scan data

Firmware Downlink

Topic	Message Type	QoS	Field	Range	Direction
/cmd_vel	geometry_msgs/Twist	best_effort + keep_last(1)	linear.x	0.15 m/s (constant)	+= forward
			angular.z	-1.20~1.20 rad/s	+= turn left, -= turn right
/cmd_ptz_angle	microros_v2_msgs/PtzAngle	reliable	pitch_deg	-90°	Tilted down toward the ground

3. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
PTZ version	☑
No-PTZ version	✗
Required peripherals	ESP32 camera (pan-tilt) + PTZ servo + T-mini Plus LiDAR

2. Start Agent + Camera + Joint Model (Terminal 1)

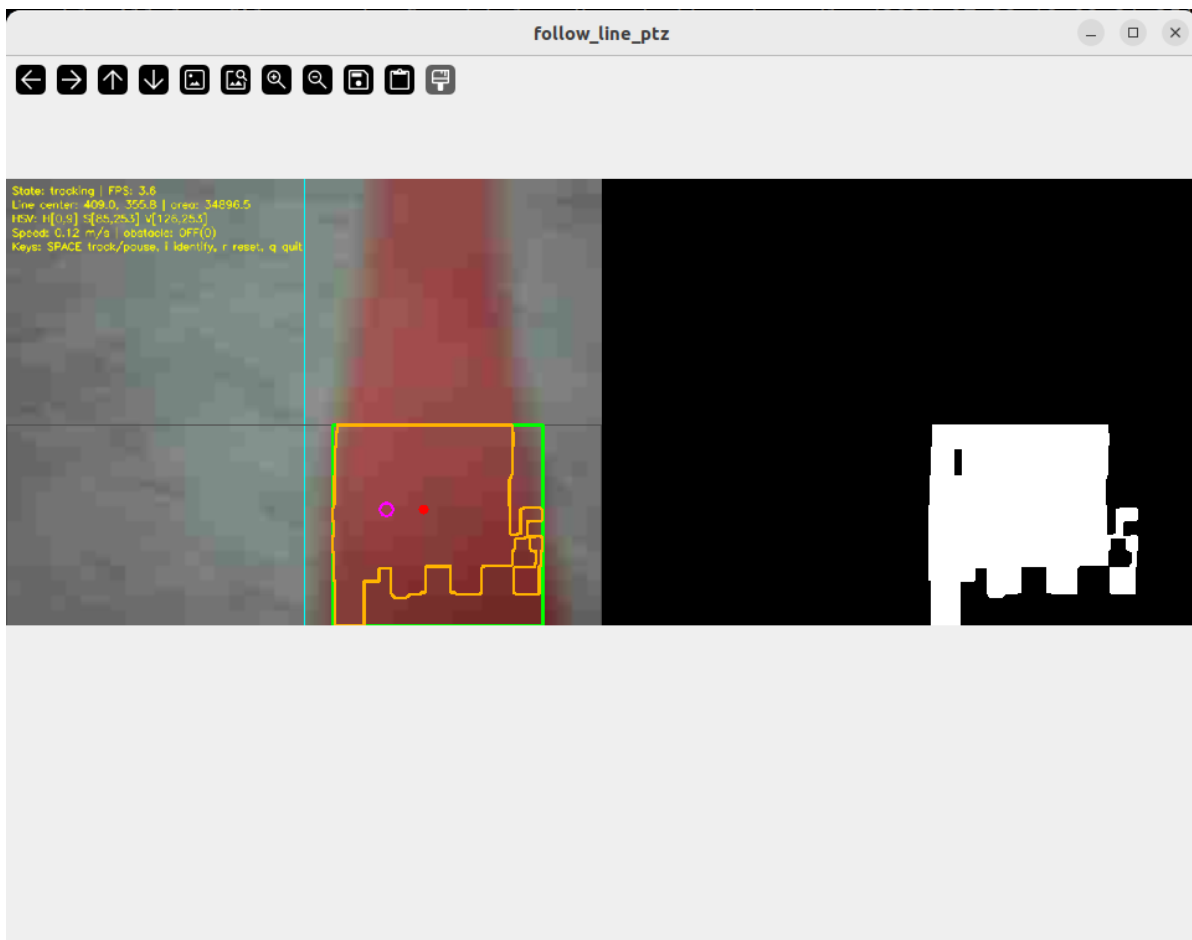
```
ros2 launch microros_v2_bringup car_base.launch.py
```

```
yahboom@yahboom:~$ ros2 launch microros_v2_bringup car_base.launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-28-15-04-46-269642-yahboom-579370
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [micro_ros_agent-1]: process started with pid [579411]
[INFO] [camera_driver-2]: process started with pid [579413]
[INFO] [robot_state_publisher-3]: process started with pid [579415]
[INFO] [joint_state_publisher-4]: process started with pid [579417]
[INFO] [ekf_node-5]: process started with pid [579419]
[micro_ros_agent-1] [1785222287.404236] info | UDPv4AgentLinux.cpp | init | running... | port: 8090
[robot_state_publisher-3] [INFO] [1785222288.847816851] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-3] [INFO] [1785222288.848013042] [robot_state_publisher]: got segment base_link
[robot_state_publisher-3] [INFO] [1785222288.848028942] [robot_state_publisher]: got segment bwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848037122] [robot_state_publisher]: got segment cam1_Link
[robot_state_publisher-3] [INFO] [1785222288.848043874] [robot_state_publisher]: got segment cam2_Link
[robot_state_publisher-3] [INFO] [1785222288.848050538] [robot_state_publisher]: got segment cam3_Link
[robot_state_publisher-3] [INFO] [1785222288.848057264] [robot_state_publisher]: got segment imu_frame
[robot_state_publisher-3] [INFO] [1785222288.848063747] [robot_state_publisher]: got segment laser_frame
[robot_state_publisher-3] [INFO] [1785222288.848070240] [robot_state_publisher]: got segment lfwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848076888] [robot_state_publisher]: got segment rfwheel_Link
[joint_state_publisher-4] [INFO] [1785222289.372349515] [joint_state_publisher]: Waiting for robot_description to be published on the robot_description
[camera_driver-2] [INFO] [1785222289.716161193] [esp32_ip_camera_publisher]: camera node started, camera_url: http://192.168.12.37:81/stream
[camera_driver-2] [WARN] [1785222292.705741414] [esp32_ip_camera_publisher]: Camera not opened, trying reconnect... (next retry in 1.0s)
[camera_driver-2] [INFO] [1785222294.072701944] [esp32_ip_camera_publisher]: IP camera stream opened successfully
```

3. Start Visual Line Tracking (Terminal 2)

```
ros2 launch microros_v2_ptz_visual_control_pkg follow_line_ptz.launch.py
```

After startup, the pan-tilt tilts down to -90° and line tracking starts automatically (auto_start_tracking:=true).



4. Operation Instructions

Key	Function
Space	tracking/paused
i / I	Recognition only (no control)
r / R	Reset
Mouse drag	HSV sampling (box-select the line to auto-update the threshold)
q / Q / ESC	Exit

5. How to Stop

Press `Ctrl+C` to stop Terminal 2 → Terminal 1 in order.

4. Experiment Observations

- **Automatic line tracking after startup:** The pan-tilt tilts down -90° toward the ground, driving along the line at 0.15 m/s
- **Line centered:** Drives straight within the 40 px deadband, with the PID outputting zero angular velocity
- **Line shifted left:** Turns right to correct (positive angular velocity); shifted right → turns left
- **Obstacle ahead:** Stops + buzzer within 0.35 m, resumes after the obstacle is removed
- **Line lost > 1.5 s:** Stops and exits automatically
- **Joystick override:** Stops + resets the PID, resumes after release

5. Program Analysis

Source code paths:

- Launch:
`~/microros_ws/src/microros_v2_ptz_visual_control_pkg/launch/follow_line_ptz.launch.py`
- Node:
`~/microros_ws/src/microros_v2_ptz_visual_control_pkg/microros_v2_ptz_visual_control_pkg/follow_line_ptz_node.py`

1. Core Node Analysis

`follow_line_ptz_node` integrates PTZ control on top of the No-PTZ version. Core features:

- **Discrete PID controller** (`DiscretePidController`): Independent P/I/D terms + integral limiting, with the `pid_scale` (1000) normalized gain (actual $P=0.022$, $D=0.005$)
- **HSV hue wraparound:** When `Hmin > Hmax` (e.g., red $H[160,10]$), merges two masks in segments
- **EMA filtering:** `target_smooth_alpha` (0.55) exponential moving average to smooth the detected center
- **Mouse sampling:** Box-select the ROI → 5th~95th percentile HSV → auto-update the threshold
- **PTZ tilt-down:** The pan-tilt tilts down to `initial_pitch_deg` (-90°) at startup, facing the camera toward the ground

Chassis control timer — PID line tracking + obstacle avoidance priority:

```
# Source file:
~/microros_ws/src/microros_v2_ptz_visual_control_pkg/microros_v2_ptz_visual_control_pkg/follow_line_ptz_node.py

def publish_chassis_control_command(self) -> None:
    if not self.is_node_running:
        return
    # Joystick override takes priority
    if self.joy_is_active:
        self.publish_zero_twist()
        return
    # Obstacle avoidance takes priority over the tracking PID
    if self.enable_obstacle_stop and self.obstacle_is_active:
        self.publish_zero_twist()
        self.publish_beep(True)
        return

    if self.tracking_state != TRACKING_STATE_TRACKING:
        return

    # Pixel error -> PID angular velocity
    line_center_x = self.filtered_line_center[0]
    pixel_error_x = line_center_x - self.image_width * 0.5

    twist_command = Twist()
    twist_command.linear.x = float(self.line_linear_speed) # constant 0.15 m/s

    if abs(pixel_error_x) <= self.line_center_deadband_pixel:
        angular_command = 0.0 # drive straight within the deadband
    else:
        pid_input = pixel_error_x / self.line_error_divisor # scaling
        pid_output = self.line_pid_controller.update(pid_input, 0.0)
        angular_command = -pid_output # reverse angular direction
        angular_command = np.clip(angular_command, -self.max_angular_speed,
self.max_angular_speed)

    twist_command.angular.z = angular_command
    self.cmd_vel_publisher.publish(twist_command)
```

2. Key Parameters

Parameter definition files:

- Launch:
~/microros_ws/src/microros_v2_ptz_visual_control_pkg/launch/follow_line_ptz.launch.py
- Node:
~/microros_ws/src/microros_v2_ptz_visual_control_pkg/microros_v2_ptz_visual_control_pkg/follow_line_ptz_node.py

Parameter	Type	Default Value	Description
<code>line_linear_speed</code>	float	0.15 m/s	Constant linear speed for line tracking
<code>max_angular_speed</code>	float	1.2 rad/s	Maximum angular velocity limit
<code>line_pid_kp</code> / <code>line_pid_kd</code>	float	22.0 / 5.0	PID gains (scaled by pid_scale=1000)
<code>pid_scale</code>	float	1000.0	PID gain normalization divisor
<code>line_error_divisor</code>	float	16.0	Pixel error scaling divisor
<code>line_center_deadband_pixel</code>	float	40.0 px	Deadband
<code>line_roi_top_ratio</code>	float	0.55	ROI top ratio
<code>target_smooth_alpha</code>	float	0.55	EMA filtering coefficient
<code>line_lost_timeout_sec</code>	float	1.5 s	Line-lost timeout
<code>enable_obstacle_stop</code>	bool	true	LiDAR obstacle avoidance switch
<code>obstacle_response_distance</code>	float	0.35 m	Obstacle response distance
<code>auto_start_tracking</code>	bool	true	Start line tracking automatically
<code>lost_line_stop</code>	bool	true	Stop after the line-lost timeout without exiting the node
<code>initial_pitch_deg</code>	float	-90.0 °	Pan-tilt tilt-down angle (facing the ground)

3. Node Description

Node	Package	Description
<code>follow_line_ptz_node</code>	<code>microros_v2_ptz_visual_control_pkg</code>	PTZ visual line tracking + obstacle avoidance, dual timers + PID + pan-tilt tilt-down

6. Frequently Asked Questions

Problem	Cause	Solution
Line tracking sways left/right	PID gains too large	Reduce <code>line_pid_kp</code> / <code>line_pid_kd</code>
Line not detected	HSV threshold mismatch	Box-select the line with the mouse to re-sample
Obstacle avoidance too sensitive	<code>obstacle_response_distance</code> too large	Reduce the response distance
Stops suddenly during tracking	<code>lost_line_stop=true</code> or HSV/LiDAR input lost	Check the input; set to <code>false</code> as needed after confirmation
Abnormal pan-tilt angle	Servo not initialized correctly	Check the PTZ power supply and servo connection
Easily leaves the line while tracking	Unstable Wi-Fi causes frame stutter, so the control parameters cannot prevent deviation however they are tuned	Use a clean, stable 2.4G Wi-Fi network and re-tune the control parameters